A Digital Twin of the Waveshare JetBot, and How Far It Can Be Trusted

With elecsim, a code-first framework for electrical simulation

Muthuphei Mukhunyeledzi

Experimental work carried out on 4 October 2026

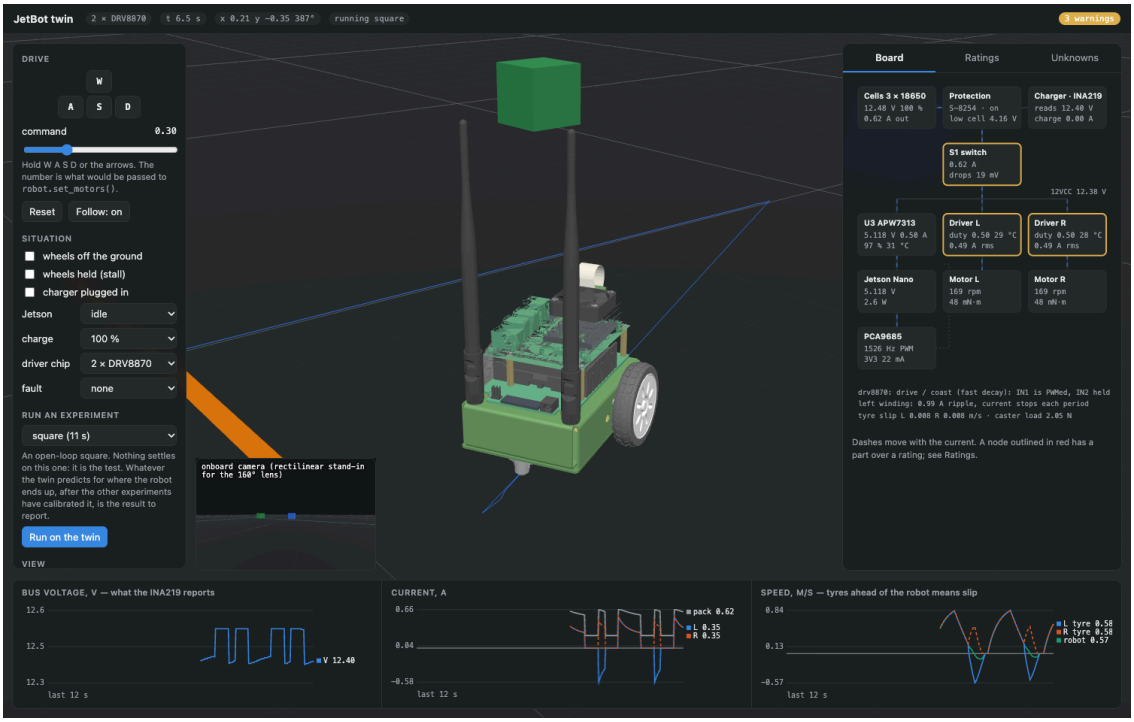


Figure 1: The live twin. The geometry is the manufacturer's own CAD; the panel on the right is the expansion board's power path with live values; the traces below are the battery voltage as the robot's own sensor would report it, the currents, and the speeds.

Repository: `jetbot-twin/` Framework: `elecsim/` Twin: `jetbot_twin/`

Summary

This project built a digital twin of a Waveshare JetBot AI Kit (Jetson Nano) and then tested it against the real robot. The twin has two parts: a model of the robot's electronics, written with a small general-purpose framework developed for the purpose, and a model of its motion. The two are stepped together, and the result is driven through the same software interface as the real robot.

The work was arranged so that the comparison with the robot would be a test and not a fit. Every number in the twin carries a record of where it came from. Predictions, with uncertainty bands, were written down before the robot was switched on. Calibration was done in stages, each from an experiment designed to isolate a few numbers, and one manoeuvre was held back from all fitting to test the result.

The main findings are these.

- **Steady motion is predicted to within a few percent.** The straight-line distance was predicted before the run to within 3 %, a two-second spin to within 6 %, and wheel speed across the whole command range to within 8 % after calibration.
- **Short, open-loop turns are not predicted.** In a driven square the twin got every side right to within a few centimetres and every corner wrong, by up to 39°. The robot itself does not repeat those corners: taken straight after a side they vary three times as much as the same turn from rest.
- **The comparison found things the design documents do not show.** The Jetson's 5 V supply falls to 4.5 V under load, below its specification, through about 0.5 Ω that is on no schematic. The board carries a different motor driver from the one first assumed, and which driver is fitted changes what a motor command means. The two motors differ by about 2.6 %.
- **Several of the project's own errors were caught by its own checks**, and are reported here as part of the result: a calibration that fitted a 10 mm wheel, a "square" that was not a square, two video-tracking methods that gave confident wrong answers, and a strip of tape left on a tyre through half the floor tests.

Contents

Summary 2
1 The idea, and how it changed 4
1.1 Where the project started 4
1.2 The two requirements 4
1.3 What "digital twin" was taken to mean 4
1.4 The approach to honesty 4
2 The robot 5
2.1 What it is 5
2.2 What the schematic says 5
3 The framework: elecsim 6
3.1 What it is for 6
3.2 The five ideas 7
3.3 Three kinds of constraint 7
3.4 Defining a part 7
3.5 How it solves 8
3.6 The library 8
3.7 Other tools in the framework 8

4 The JetBot twin 9
4.1 The circuit 9
4.2 The software interface 9
4.3 The dynamics 9
4.4 Stepping the two together 9
4.5 The geometry 9
4.6 The viewer and the blueprint 10
5 The repository 11
5.1 Layout 11
5.2 How to run it 11
5.3 How a comparison works 12
6 Method 12
6.1 What the robot can and cannot report 12
6.2 The experiments 12
6.3 Predictions made beforehand 13
6.4 A rehearsal before the real thing 13
6.5 The agent on the robot 13
6.6 Reading the videos 13
7 Results 13
7.1 The probe 13
7.2 Rest, and a known load step 14
7.3 Which driver chip 15
7.4 Wheels off the ground 15
7.5 Stall 17
7.6 On the floor: straight and spin 17
7.7 The held-out square 17
7.8 Why the corners: the short turn on its own 18
7.9 A confound found late: tape on a tyre 20
7.10 A second square, tape off, timing tight 20
7.11 Summary of the comparison 22
8 What changed in the twin 22
9 Mistakes, and how they were caught 22
9.1 In the method 23
9.2 In the agent on the robot 23
9.3 In measuring from video 23
9.4 In the model's geometry 24
9.5 At the bench 24
10 What the twin is good for 24
10.1 Where it can be trusted 24
10.2 Where it cannot 24
10.3 The general lesson 24
11 Limits of this work 25
12 What to do next 25
Appendix A: the experiment definitions 26
Appendix B: where the evidence is 26
Appendix C: sources 26

1 The idea, and how it changed

1.1 Where the project started

The original research plan was a RoboCup Small Size League robot, designed as a parametric CAD model with a rule checker, an electrical model and a browser "digital twin" for looking at the design. The supervisors liked the idea of a digital twin and of simulating the circuit. By the time that design was ready it was too late to order parts.

The research was therefore redirected to a robot that already existed on the bench: a Waveshare JetBot AI Kit built around a Jetson Nano. The question became sharper as a result. A twin of a robot that does not exist can only be judged by whether it looks right. A twin of a robot that does exist can be judged by whether it is right.

1.2 The two requirements

Two requirements came with the redirection.

1. **Build a digital twin of the JetBot and see how it compares to the real thing.** The deliverable is the comparison, not the simulator.
2. **The electrical simulation must be a framework.** Code-based, so that someone else can clone the repository and define their own components and their own constraints, for a different robot or a different circuit.

1.3 What "digital twin" was taken to mean

The term is used loosely. In this project it means four things, all of which were built.

Aspect	What it is here
Geometry	The manufacturer's CAD assembly, converted and named, drawn in a browser
Physics	The board's circuit and the robot's dynamics, solved together in time
Interface	The same calls as the real robot: `robot.set_motors(left, right)`
Sensors	The twin reports what the robot's own sensors would report, with their resolution and their blind spots

The last of these matters more than it seems. A twin can only be compared with a robot through that robot's sensors. If the twin reports the true battery current and the robot has no sensor for battery current, there is nothing to compare.

1.4 The approach to honesty

A simulation is easy to make agree with a robot after the fact. Four habits were adopted to stop that.

- **Provenance.** Every number in the twin is tagged as taken from a datasheet, measured, or assumed. The assumed ones are listed in every report the software prints.
- **Predictions before measurements.** The twin's predictions, with error bands from the stated uncertainty in its assumed numbers, were saved to disk before the robot was powered.
- **Staged calibration.** Numbers were fitted a few at a time, each from an experiment in which only those numbers could explain the result.
- **A held-out test.** One manoeuvre, an open-loop square, was used to fit nothing.

2 The robot

2.1 What it is

The JetBot AI Kit is a two-wheel differential-drive robot. A folded aluminium chassis carries two geared motors, a ball caster front and rear, an expansion board with three 18650 lithium-ion cells, and a Jetson Nano developer kit above it with a wide-angle camera on a hinged mount at the front.

Two documents from the manufacturer were the starting point: the expansion board's schematic, and a STEP file of the whole assembly. Both are in `reference/`.

2.2 What the schematic says

The board's circuit was transcribed from the schematic into the twin, component by component, keeping the schematic's reference designators. The power path is:

```
cells -- holder -- protection --+-- EB+ -- S1 -- 12VCC --+-- buck -- 5V -- Jetson
3x18650           (S-8254AA,    |      (switch)          +-- driver -- motor L
                  4x AO4407A)   R16 0.1 ohm              +-- driver -- motor R
                                |  (read by the INA219)
                             DC jack (charger)
```

Reading the schematic closely produced seven observations that shaped everything afterwards.

1. **Two alternative motor drivers.** The board has footprints for one TB6612FNG or for two DRV8870s, wired to the same motor connectors and to different outputs of the PWM chip. One or the other is fitted.
2. **They drive the motor differently.** The TB6612FNG shorts the motor between pulses (slow decay). The DRV8870, as the software drives it, switches everything off (fast decay). Figure 2 shows the consequence.
3. **The battery monitor does not measure the load.** The INA219′s shunt is between the charger jack and the battery rail. It reads charger current and rail voltage. The robot has no sensor for what it draws.
4. **The motors are fed the full pack voltage**, up to 12.6 V.
5. **All current passes through one slide switch**, a part whose common rating is 0.3 A.
6. **The 5 V rail is set to 5.118 V**, from the feedback divider.
7. **The PWM frequency is 1526 Hz**, not the 1600 Hz the library asks for, because of the chip's prescaler.

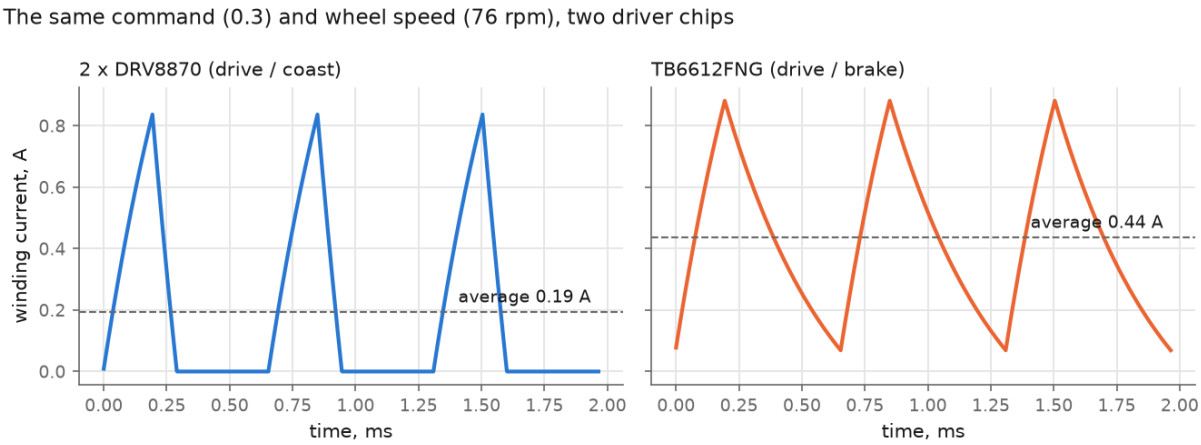


Figure 2: Winding current over three PWM periods at the same command and the same wheel speed, for the two possible driver chips. Simulated with the framework's switching H-bridge model.

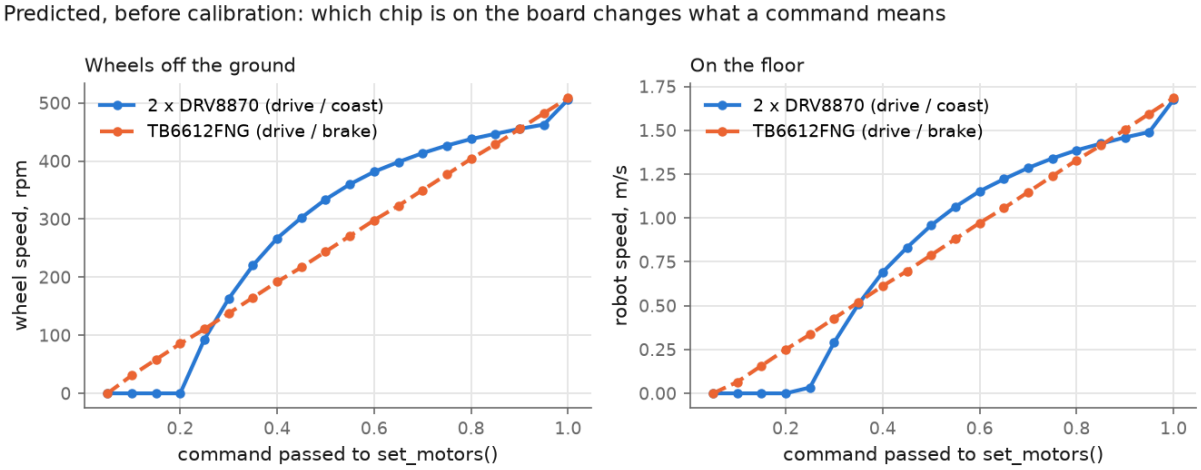


Figure 3: Predicted speed against command for the two chips, before any measurement. The drive-and-coast chip has a dead band; the drive-and-brake chip is nearly linear from the start.

The difference in Figure 3 is the twin's first testable prediction: which chip is on the board decides whether the robot moves at a command of 0.1.

3 The framework: elecsim

3.1 What it is for

`elecsim` lets a circuit be described in Python, constrained, and simulated. It has no knowledge of the JetBot. It needs Python and numpy.

A design is a Python file with a function `build()` that returns a circuit. That is the whole contract.

```
from elecsim import Circuit, Scenario, assumed, datasheet
from elecsim.lib import Cell, Resistor, Switch

def build():
    c = Circuit("torch")
    cell = c.add(Cell("cell", capacity_ah=datasheet(2.6, "cell datasheet"),
                      r0=assumed(0.06, "typical; not measured")))
    sw   = c.add(Switch("sw", r_on=0.03, i_max=1.0))
    lamp = c.add(Resistor("lamp", r=6.0, p_max=3.0))
    c.connect("BAT", cell.p, sw.a)
    c.connect("LAMP", sw.b, lamp.a)
    c.ground(cell.n, lamp.b)
    c.require("the lamp gets at least 2 W", lambda r: r.min("lamp.heat") >= 2.0)
    return c

SCENARIOS = [Scenario("ten minutes on", duration=600, dt=1.0)]
```

3.2 The five ideas

Idea	Meaning
Component	A part: its pins, its numbers, and the equations that relate them
Circuit	Components with their pins joined into named nets
Constraint	Something that must hold; three kinds, below
Scenario	A situation to put the circuit in
Report	Every constraint in every scenario, pass or fail, with the unverified numbers it rests on

3.3 Three kinds of constraint

They are separate because they answer different questions.

Kind	Question	How it is written
Rule	Is it wired sensibly?	A function of the circuit, run without simulating. Built in: unconnected pins, two supplies on one net, a 5 V output into a 3.3 V input, and others
Limit	Is any part over a rating?	Declared on the component, from its datasheet. Watched on every step. Can require the excess to last, which separates a peak rating from a continuous one
Requirement	Does the design do its job?	A question asked of a finished run: “the 5 V rail stays above 4.75 V”

3.4 Defining a part

A part is a class. Its pins and numbers are declared; `stamp` adds its equations; `measure` says what can be observed about it.

```python
class Resistor(Component):
    a = Port()
    b = Port()
    r = Param(unit="ohm", min=0, doc="resistance")

    def stamp(self, s):
        s.G(self.a, self.b, 1.0 / self.r)

    def measure(self, s):
        v = s.vd(self.a, self.b)
        return {"v": v, "i": v / self.r, "heat": v * v / self.r}
```

Whatever `measure` returns becomes a named signal, and signals are what limits watch, what requirements ask about and what is recorded.

3.5 How it solves

Modified nodal analysis. The unknowns are the voltage of every net plus any extra unknowns components ask for. Each Newton iteration, every component adds its linearised equations; the system is solved; this repeats until the answer stops moving. Time stepping is backward Euler, with the step halved automatically when a solve fails.

Two refinements were added because real use demanded them.

- When plain Newton fails, the solve is retried with the voltages only allowed to creep. This lets a part written naively by a user converge.
- When there is no solution at all, because something has lost its supply, regulators are allowed to switch themselves off and the rest is solved. Without this, simulating an unplugged battery simply failed.

3.6 The library

Passives, sources and loads, diodes and MOSFET switches, a lithium cell with an open-circuit-voltage curve and state of charge, a multi-cell pack protector, a buck converter whose efficiency follows from its losses, a DC motor, a switching H-bridge, a PWM controller with the PCA9685′s registers, and measurement chips with their real resolution.

3.6.1 Two models of the same motor drive

A PWM-driven motor can be simulated edge by edge, which needs steps of microseconds, or averaged over the PWM period, which needs steps of milliseconds. The framework has both. The averaged one solves the periodic waveform in closed form, so it is correct in discontinuous conduction and in both decay modes. The test suite holds the two to within 3 % of each other. The twin uses the averaged model and runs at about three times real time.

3.7 Other tools in the framework

- **Fault analysis.** Every open and short the components can suffer is applied in turn and the scenario rerun, giving a table of which requirements each fault costs.
- **Sweeps, sensitivities, Monte Carlo.** These work on any function from parameters to numbers, so they apply to a circuit or to a whole twin.
- **A command line.** `python -m elecsim new`, `check`, `fmea`, `parts`, `show`.

4 The JetBot twin

4.1 The circuit

`jetbot_twin/electrical.py` is the schematic in code: three cells, the holder's contact resistance, the protection circuit, both shunts, the INA219, the switch, the buck converter, the Jetson, the PWM chip, and the two motor drives. One part, the Jetson Nano, did not exist in the library and was written for this design, which is the intended way to use the framework.

Run on its own, the circuit is checked against six scenarios from idle to "both motors stalled, Jetson at full power, pack nearly empty".

4.2 The software interface

The twin's `Robot` has the same methods as the real one. Underneath, it does what the real `Motor` class does: scales the value to 0–255, multiplies by 16, and writes the same PCA9685 registers. The program that runs on the real robot makes the identical writes. A test asserts that the two produce the same register values for a range of commands.

So the twin and the robot are compared on the same register values, not on "a command of 0.3" in the abstract.

4.3 The dynamics

State: position, heading, forward speed, yaw rate, and the two wheel speeds.

- **Each wheel** has the rotor's inertia reflected through the gearbox, and friction that is part constant and part proportional to speed.
- **Each tyre** transmits a force that grows with its slip speed and saturates at the friction limit. The wheels are not assumed to roll without slipping.
- **The caster** is a ball that drags, opposing its own motion over the floor.
- **The body** has its centre of mass behind the axle, and the loads on the wheels and caster shift with acceleration.

Left out, and stated as such: sideways tyre slip, gearbox backlash, any difference between the two motors, floor texture.

4.4 Stepping the two together

Each step, the wheel speeds go into the circuit as back-EMF, the circuit is solved for the motor currents, the currents become torques, and the dynamics move the robot. The step is 1 to 2 ms.

4.5 The geometry

The manufacturer's STEP file was measured to give the track, the caster positions and the body dimensions, and converted to a browser model with named parts. Three things in the model are not the manufacturer's.

- **The camera mount.** The CAD has a fixed bracket; this robot has a hinged one. It was rebuilt from photographs, in six attempts (see Section 9).
- **The cells.** The kit ships without them.
- **The ribbon cable**, redrawn between its two connectors after the camera was moved.

Because that geometry is hand-made, the build checks it before writing anything: nineteen checks, including that the mount passes through no other part, that the camera can tilt on its hinge, and that the cells fit their holder.

4.6 The viewer and the blueprint

`python -m jetbot_twin serve` runs the twin in real time and opens two pages. Nothing is computed in the browser; every number on either page comes from the Python twin.

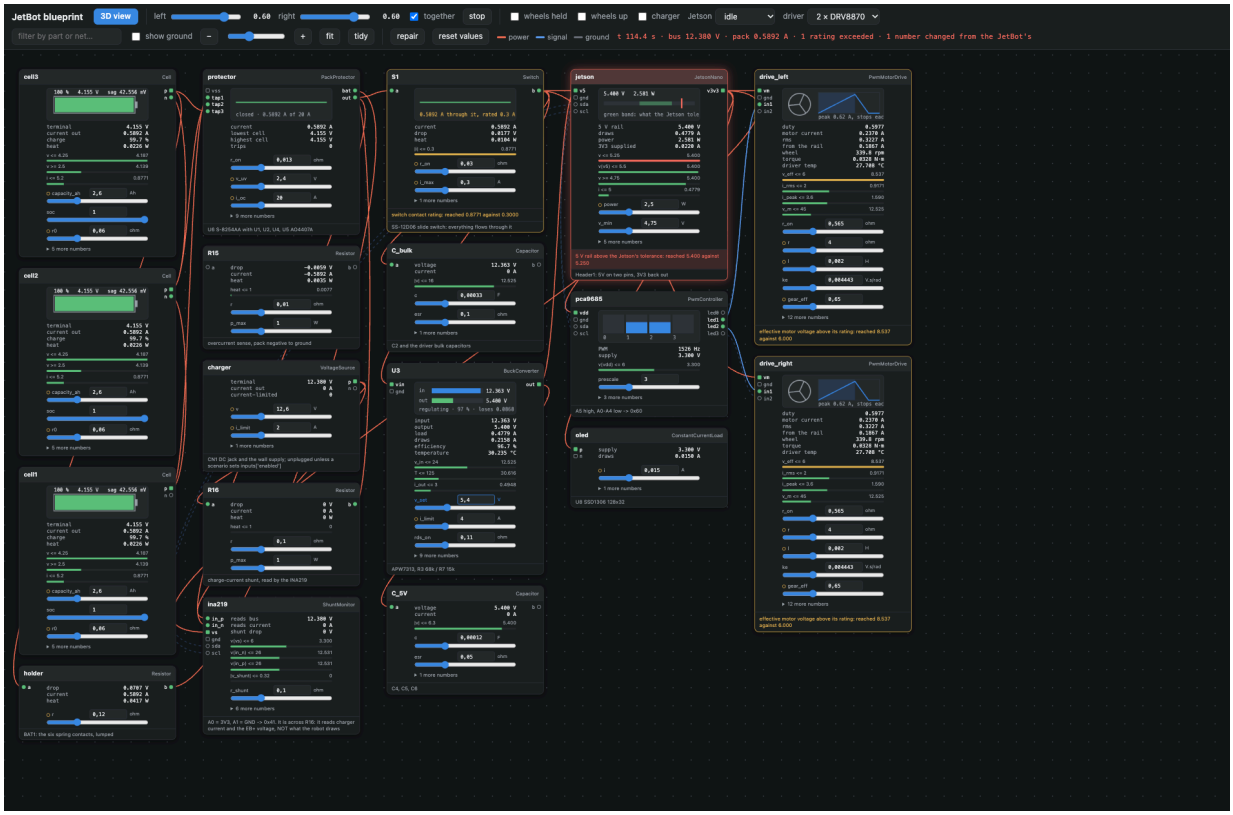


Figure 4: The blueprint page. Every component of the board is a node with its pins, a live instrument and its numbers. Every number starts at the JetBot's own value and can be changed. Here the 5 V set point has been raised to 5.4 V and the Jetson node reports its supply above tolerance.

5 The repository

5.1 Layout

Path	Contents
`elecsim/`	The framework. `core.py` (vocabulary), `solver.py`, `constraints.py`, `erc.py` (rules), `design.py` (scenarios), `faults.py`, `analysis.py`, `pwm.py`, `lib/` (parts)
`jetbot_twin/hardware.py`	**Every number, and where it came from. The file to edit**
`jetbot_twin/electrical.py`	The board, component by component
`jetbot_twin/dynamics.py`	Tyres, casters, mass
`jetbot_twin/twin.py`	The two stepped together; the `Robot` interface
`jetbot_twin/experiments.py`	The experiments, shared by twin and robot
`jetbot_twin/compare.py`, `calibrate.py`, `predict.py`	Twin against robot; staged fitting; predictions with uncertainty
`jetbot_twin/server.py`, `web/`	The live viewer and blueprint
`robot/agent.py`	The one file that runs on the JetBot
`tools/`	CAD conversion, figures, and the video-measurement tools
`data/real/`	Logs, videos and hand measurements from the robot
`results/`	Predictions, fits, comparison figures
`docs/`	`FRAMEWORK.md`, `ELECTRICAL.md`, `MODEL.md`, `VALIDATION.md`, `RESULTS.md`
`tests/`	60 tests

5.2 How to run it

```
python3 -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"
pytest                                  # 60 tests, about 30 s

python -m jetbot_twin serve             # live 3D twin; press B for the blueprint
python -m jetbot_twin info              # the robot as the twin knows it
python -m jetbot_twin assumptions       # what has not been measured
python -m jetbot_twin check             # the board against every rating
python -m jetbot_twin run square        # one experiment on the twin
python -m jetbot_twin predict           # predictions with error bands

python -m elecsim new my_board          # a starter design of your own
python -m elecsim check my_board.py
```

With the robot powered:

```
python -m jetbot_twin real probe --host jetbot@<ip>
python -m jetbot_twin real run straight --host jetbot@<ip> --session 2026-10-04
python -m jetbot_twin compare --session 2026-10-04
python -m jetbot_twin calibrate --session 2026-10-04
```

5.3 How a comparison works

An experiment is a list of (seconds, left, right) commands. The same definition is sent to the robot, where the agent executes it and logs what the robot can sense, and run on the twin. Both produce a log with the same columns. `compare` replays the twin on the robot's own conditions: the commands at the times they were actually sent, the Jetson power the robot measured, and the state of charge that matches the robot's resting voltage.

6 Method

6.1 What the robot can and cannot report

The robot measures	It has no sensor for
Battery rail voltage (INA219, 4 mV steps)	Its own load current
Charger current (the same chip)	Motor current
Power into the Jetson (the Jetson's own monitor)	Wheel speed
Temperatures, CPU load	Position or heading

Everything in the right-hand column had to be measured from outside. In practice that meant a tape measure and a phone camera, with software written to read the videos (Section 6.6).

6.2 The experiments

Each was designed so that only a few of the twin's unknowns could explain its result.

Experiment	Set-up	What it settles
`rest`	bench	idle power, state of charge
`jetson_load`	bench	pack resistance: the Jetson's CPUs are loaded, and since the Jetson measures its own power this is a **known** load step
`wheels_up_sweep`	wheels in the air	the motor's speed constant, friction, and which driver chip
`stall`	wheels held	winding resistance: no motion means no back-EMF
`straight`	floor	drag on the floor; left-right mismatch
`spin`	floor	caster friction
`square`	floor	nothing: this is the held-out test
`short_turns`	floor	how repeatable the square's corner is

6.3 Predictions made beforehand

Forty robots were drawn from the stated uncertainty in the unverified numbers, each run through the experiments, and the 5th to 95th percentile band recorded. Two findings came out of this before any measurement.

- The bands were wide: a factor of two on distance, because nobody had yet measured what the motors were.
- The open-loop square could not be predicted at all. The band on its final heading covered nearly the whole circle.

6.4 A rehearsal before the real thing

Before the robot was powered, the whole procedure was run against a stand-in: a second twin with nine of its numbers moved by 10 to 45 %, read through noisy sensors and a tape measure that rounds. The purpose was to find out whether the procedure recovers known answers.

It found two faults in the procedure itself (Section 9). After they were fixed it recovered every altered value to within 9 % and reduced the error on the held-out square from 46 cm to 4 cm.

> The rehearsal is not data. Every log it produces is marked `source = synthetic`, and nothing from it appears in the results below.

6.5 The agent on the robot

`robot/agent.py` is one file with no dependencies beyond what the robot's image has. It executes an experiment, logs the robot's sensors at 50 Hz, and stops the motors when it finishes, fails, or loses its connection. It was revised four times, each because the real robot showed up a fault in the previous version (Section 9).

6.6 Reading the videos

Three tools were written.

Tool	Method
`wheel_tape.py`	A strip of tape on the hub is a once-per-turn marker. The wheel's angle is read in every frame of a slow-motion video. The true frame rate (240.3 fps) is recovered from the experiment's own timing
`spin_video.py`	The robot's heading is the direction from the centre of its green chassis to its lit blue display: a marker with no symmetry
`path_video.py`	The floor's tile joints, 50 cm apart, fix the map from picture to floor including perspective. The robot is found as a shape against the empty floor

In each case the first method tried was wrong, and wrong with confidence. The lesson was the same each time: use a marker that cannot be confused with itself.

7 Results

7.1 The probe

The first contact with the robot only read from it.

Item	Found
Computer	Jetson Nano developer kit, JetPack 4.5, Python 3.6.9
I²C devices	0x3C display, 0x41 INA219, 0x60 PCA9685, as the schematic's address strapping implies
PWM chip	Asleep, prescaler at its power-on default
Pack	10.27 V at rest: nearly flat
Software	The `jetbot` package lives in a container; the host has no Python I²C library

The last row forced the first change to the agent: it now talks to the I²C device directly.

7.2 Rest, and a known load step

The Jetson's CPUs were loaded for 20 s. Its own monitor showed its draw rise from 1.12 W to 5.03 W.

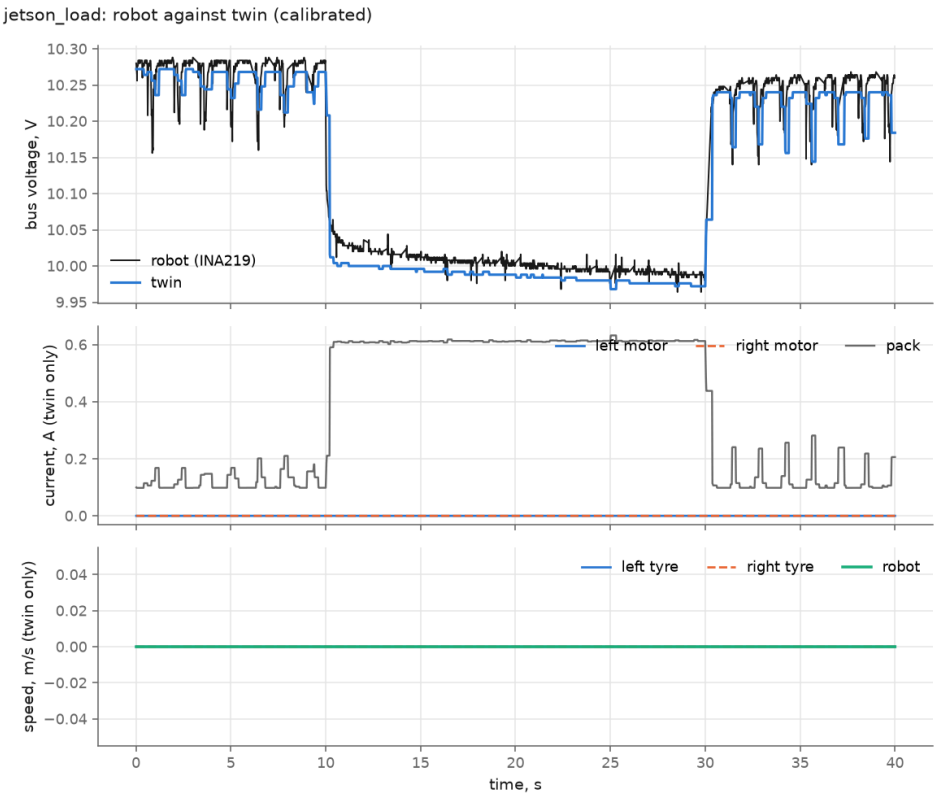


Figure 5: Battery rail voltage through the load step. Black: the robot. Blue: the twin after two numbers were fitted. The small periodic dips are the Jetson's own activity, which the twin is fed from the robot's log.

Quantity	Robot	Twin as designed	Twin fitted
Rail voltage lost to the step (flat pack)	266 mV	about 150 mV	266 mV
Rail voltage error, RMS		80 mV	30 mV
5 V at the Jetson, loaded	4.50 V	5.118 V	4.49 V
Jetson idle power	1.1 W	2.5 W	from the log

7.2.1 Finding: the Jetson's supply sags below its specification

The twin, built from the schematic, held the Jetson's supply at 5.118 V whatever the load. The robot's falls to 4.50 V with only its CPUs loaded, and to 4.57 V on a charged pack. That is about

0.5 Ω between the regulator and the point where the Jetson measures its supply: header pins, carrier traces and the carrier's input switch, none of which is on the expansion board's schematic. The Jetson is specified for 4.75 V and above.

The twin's earlier report that "the Jetson is safe in every scenario" was wrong, because of something the twin did not contain. The resistance is now a parameter of the model.

> This rests on the Jetson's own reading of its input voltage, which was not checked against a multimeter.

7.2.2 Finding: pack resistance depends strongly on charge

Condition	Fitted resistance per cell
Assumed	0.060 Ω
Measured, pack nearly flat	0.125 Ω
Measured, pack charged	0.0575 Ω

The assumption was right for a charged pack. The resistance roughly doubles as the pack empties. The twin has one number and no dependence on charge: a known gap.

7.3 Which driver chip

The robot was not to be opened, so the chip was identified electrically. The two possible chips hang off different PWM channels; the agent drove one set at a time, wheels off the ground, and watched the rail.

Channels driven	Rail dipped by
0 to 3 only (DRV8870 inputs)	0 mV
8 to 13 only (TB6612FNG pins)	204 mV

The board carries the TB6612FNG. This corrected an assumption: the presence of the INA219 had been taken to mean the later, DRV8870 revision.

7.4 Wheels off the ground

Ten commands from 0.1 to 1.0, two seconds each, filmed in slow motion with tape on the hub.

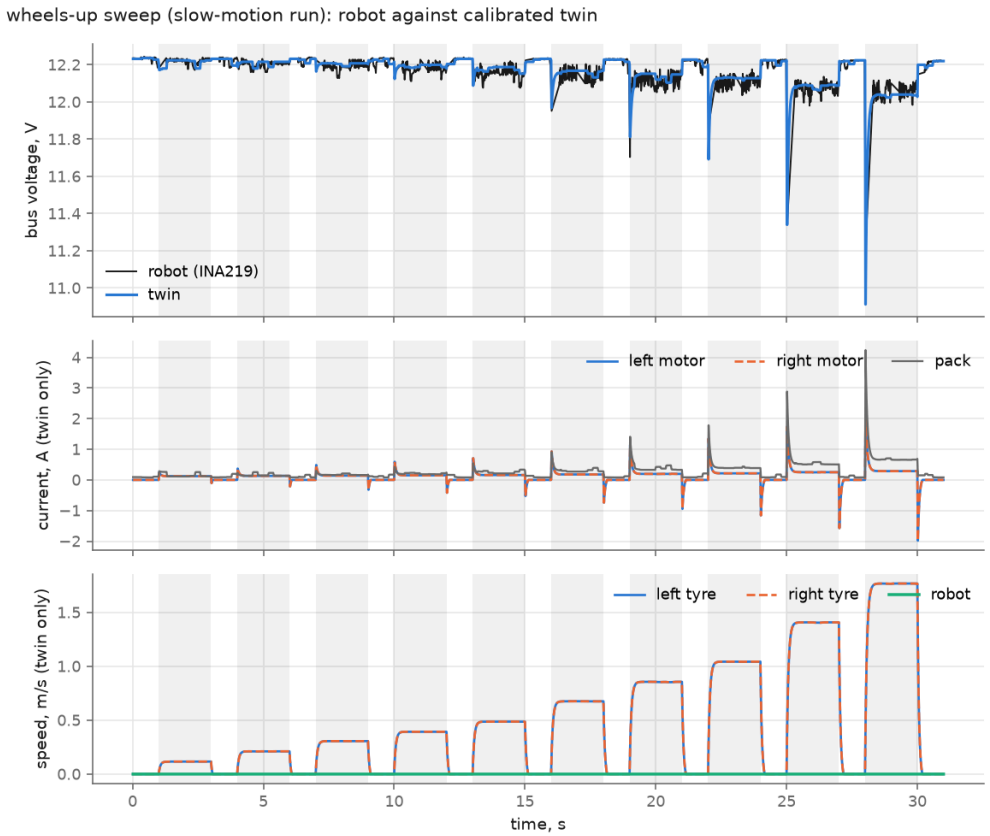


Figure 6: The wheels-up sweep. Top: rail voltage, robot and calibrated twin. Middle and bottom: what only the twin can show.

Command	Robot, rpm	Twin as designed	Twin calibrated
0.10	43	30	34
0.20	99	83	90
0.30	157	133	143
0.40	213	187	198
0.50	268	238	252
0.60	319	290	307
0.80	421	394	414
1.00	484	496	519
RMS error		22 rpm (14 %)	16 rpm (8 %)

7.4.1 The predictions held

Quantity	Predicted, 5 to 95 %	Measured
Lowest command that turns a free wheel	0.10	0.10
Free wheel at 0.5	199 to 284 rpm	268 rpm
Free wheel at 1.0	413 to 596 rpm	484 rpm
Shape of speed against command	a straight line from 0.1	straight from 0.1 to 0.8
When the command returns to zero	brakes	stops in 0.09 to 0.20 s

7.4.2 Other results from the sweep

- **The motors are ordinary 6 V units**, about 6 % faster than the manufacturer's figure: 255 rpm at 6 V against a stated 240. At a command of 1.0 they run at twice their rated voltage.
- **The dynamics agree without being fitted.** At a command of 0.5 the wheel reaches 63 % of its speed in 83 ms and stops in 162 ms. The twin gives 80 ms and 140 ms.
- **The PWM frequency is about 1560 Hz**, read from the motor's whine on the soundtrack: the chip's oscillator runs about 2 % fast.

7.4.3 What the twin still gets wrong here

The robot's speed stops rising in proportion at the top. From 0.6 to 0.8 it gains 102 rpm; from 0.8 to 1.0, only 63. At full command the robot is slower than the twin and also draws less current. A loss growing with the square of speed was tried and did not help, because it would raise the current. This is unexplained.

7.4.4 Two runs, and the floor they set

The sweep was run twice. The voltage dips in the two runs differ from each other by 24 mV RMS. The calibrated twin's dips differ from the robot's by 21 mV RMS: as close as the robot is to itself.

7.5 Stall

Both wheels held, command 0.5 for 1.5 s. The rail lost 486 mV; the twin as designed gave 443 mV. Fitting the winding resistance gives 3.45 Ω against an assumed 4.0 Ω. It is inferred through the pack resistance, not metered.

7.6 On the floor: straight and spin

Both predictions were saved before the runs, with the twin fitted only from the wheels-in-the-air tests.

Quantity	Twin predicted	Robot
Straight, 0.5 for 2 s: distance	1.47 m	1.43 m
Spin, ∓0.4 for 2 s: rotation	1095° (3.04 turns)	1033° (2.87 turns)
Spin: steady rate	556°/s	504°/s
Spin: ended this far from its start	1 cm	1 cm

Distance to within 3 %, rotation to within 6 %. The twin was a little optimistic both times and both times the robot drew more current than expected, which is one cause: more drag on the floor than assumed. Fitting the twin's two drag numbers brings both to within 1 %.

> These two runs were made with a strip of tape on the left tyre (Section 7.9). They were repeated without it.

7.7 The held-out square

Four sides at 0.5 for 1.0 s, each followed by a left turn at ∓0.3 for 0.29 s, which is a quarter-turn in the calibrated twin. Nothing was fitted to it.

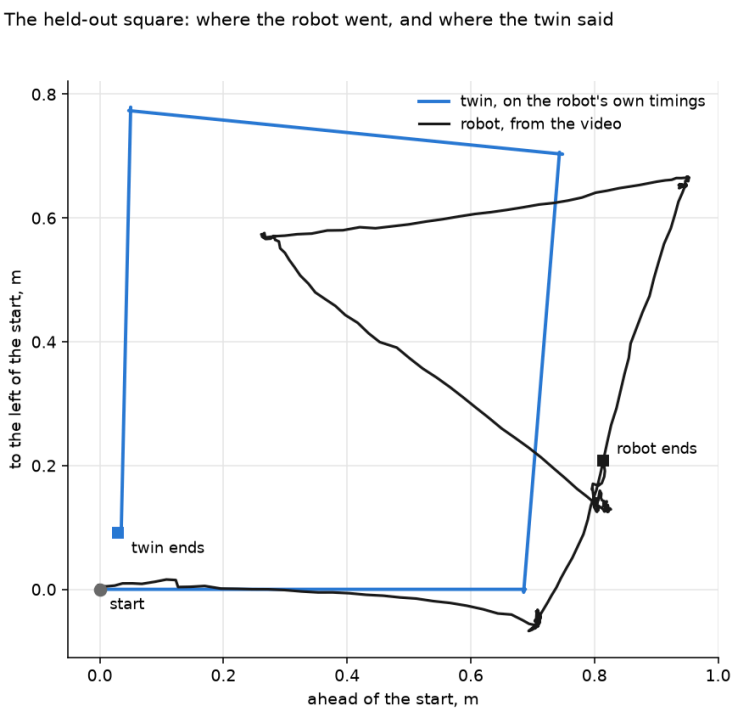


Figure 7: The first square. The twin is the model replayed on the timings the robot actually achieved.

Quantity	Twin	Robot
The four sides	0.69, 0.71, 0.71, 0.69 m	0.70, 0.72, 0.69, 0.69 m
The corners	85°, 89°, 94°	78°, 116°, 133°
Ends this far from its start	0.10 m	0.84 m

The twin got the straights and missed the turns. Every side is within 3 cm. The corners are out by 7°, 27° and 39°, and that is the whole of the miss.

7.8 Why the corners: the short turn on its own

7.8.1 Ten turns, and a discovery about timing

The square's corner was repeated ten times from rest.

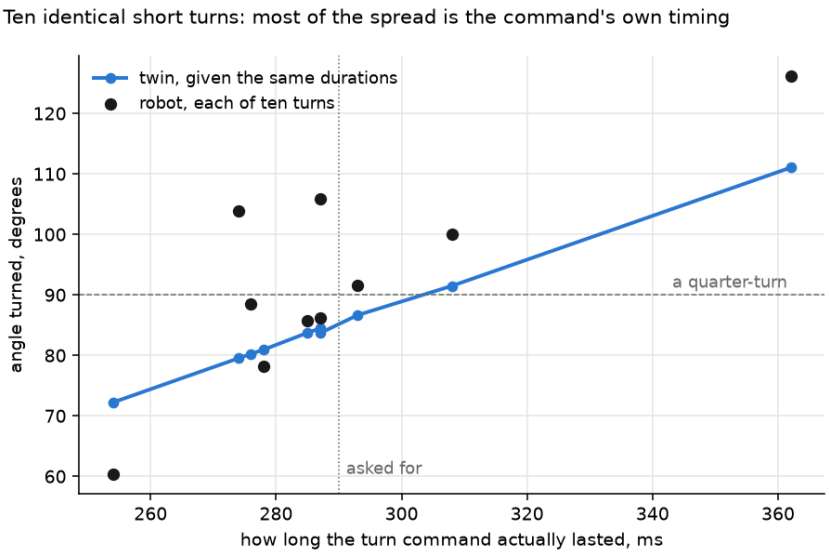


Figure 8: Ten identical turn commands. The angle turned follows how long the command actually lasted.

The turns ranged from 60° to 126°. But the command, asked to last 290 ms, had lasted anywhere from 254 to 362 ms, and the angle followed the duration (correlation 0.82). Sending a motor command is fourteen register writes over I²C from Python, and that takes tens of milliseconds, not the same every time.

> The stock `jetbot` library sends commands the same way. This is how the robot behaves in ordinary use, not a quirk of the test.

The agent was reworked so that the writes which start and stop the wheels land on time. The standard deviation of a 290 ms turn's duration fell from about 29 ms to 2.2 ms.

7.8.2 Twenty-four turns, with the timing fixed

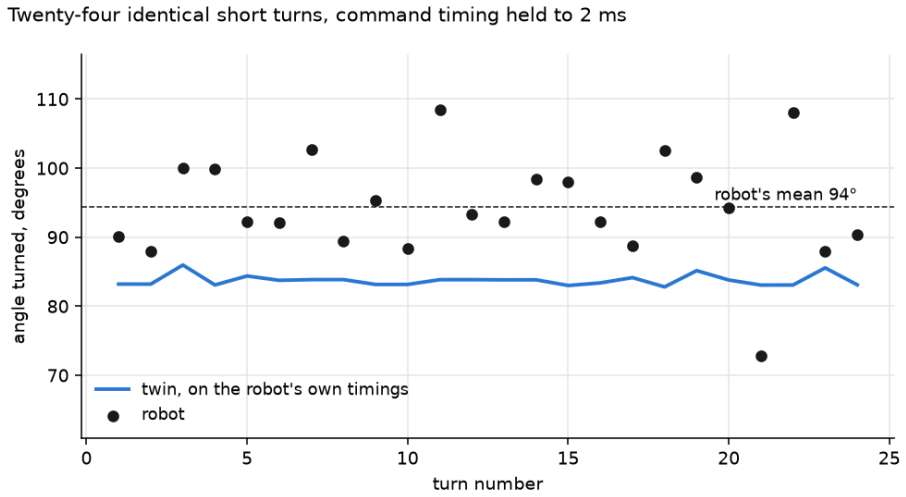


Figure 9: Twenty-four identical short turns with command timing held to 2 ms.

Quantity	Robot	Twin
Mean turn	94.3°	83.7°
Standard deviation	7.6° (8 %)	0.8°
Range	73° to 108°	
Trend through the run	none	

- **The bias is real.** The robot turns 10.6° further than the twin, 12.6 % more. Over 24 turns that is 6.9 standard errors. With ten turns and loose timing it had been two, and was not relied on.
- **The robot repeats a short turn to about ±8°.** That is a floor under any model.
- **No single number in the twin explains the bias.** Each plausible one was fitted to these turns and then checked against the spin and the straight run.

Number moved	Result
Breakaway friction	No effect
Rotor inertia	Runs to its limit and is still 2.5° short; would also break the wheels-up spin-up time, which matched
Inertia about the vertical axis	Runs to its limit and is still 6° short
Caster friction	Matches the turns exactly, but the 2 s spin goes to 1084° (robot 1033°) and the straight run to 1.48 m (robot 1.43 m)

The caster's drag would have to be lower in the first third of a second of a turn than in a sustained spin. The twin has one value for both. This is a statement about the model's form, not its numbers, and it is a hypothesis: nothing measured tests it directly.

7.9 A confound found late: tape on a tyre

A strip of tape put on the left tyre for the slow-motion wheel video stayed on through every floor test up to this point. The straight run and the spin were repeated without it.

Quantity	Tape on	Tape off	Twin, tape-off run
Straight: distance	1.43 m	1.47 m	1.45 m
Straight: sideways drift	5 to 10 cm left	over 25 cm right	0
Spin: rotation	1033°	1037°	1043°

- **The tape barely touched distance or rotation**, so the floor-drag numbers fitted to the tape-on runs stand.
- **It reversed the drift, and that exposes something real.** Without the tape the robot veers 25 cm right in 1.47 m: a difference of about 2.6 % between the two wheels. The tape had been slowing the left wheel enough to hide a slower right one. The twin's motors are identical and it predicts no drift.

7.10 A second square, tape off, timing tight

Sides shortened to 0.6 s to fit a 2 × 2 block of tiles under a phone held still. All four turns lasted 285 to 292 ms.

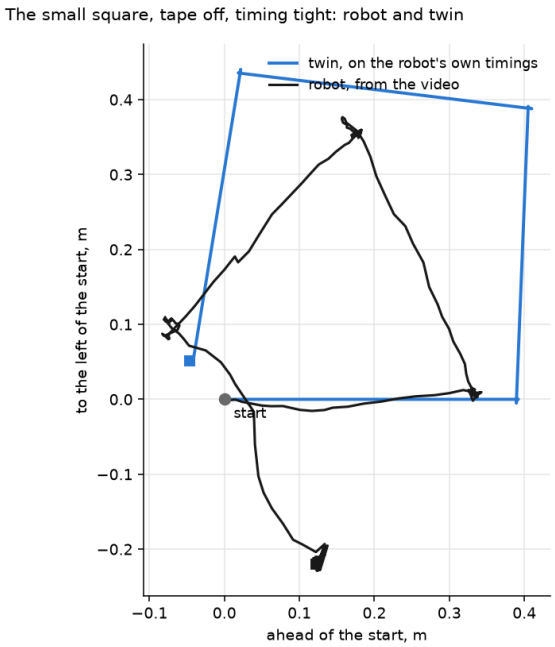


Figure 10: The smaller square, without the tape and with command timing fixed.

Quantity	Twin	Robot
The four sides	0.39, 0.40, 0.39, 0.40 m	0.33, 0.39, 0.37, 0.39 m
The corners	88°, 85°, 88°	113°, 112°, 75°
Ends this far from its start	0.07 m	0.25 m

The same verdict, now without the tape or the timing error to blame.

7.10.1 Six corners against twenty-four turns

Turn	Mean	Standard deviation
The same command, from rest (24 turns)	94°	8°
The same command, straight after a side (6 corners)	105°	23°

A corner taken straight after a side is three times as variable as the same turn from a standstill. The command is identical; what differs is the state the robot arrives in. Two candidates the twin has no model of are gearbox backlash (after driving forward, a left turn asks the left gear train to reverse through its slack while the right one pushes at once) and whatever the robot is still doing 0.6 s after braking. Which it is has not been established.

7.11 Summary of the comparison

Test	Twin against robot	Predicted beforehand?
Rail voltage through a load step	to the sensor's noise	fitted
Wheel speed, wheels up	within 8 %	bands held
Spin-up and braking time	80 against 83 ms; 140 against 162 ms	yes, never fitted
Straight-line distance	within 3 %	yes
Two-second spin	within 6 %	yes
Square: the four sides	within 3 cm each	yes
Short turn from rest	13 % low; robot scatter ±8°	no
Square: the corners	wrong by up to 39°	no
Straight-line drift	not modelled	no

8 What changed in the twin

Number	Was	Now	From
Driver chip	DRV8870 (a guess)	TB6612FNG	driver-chip test
5 V path resistance	absent	0.49 Ω	load step
Cell resistance	0.060 Ω	0.0575 Ω	load step, charged pack
Motor speed at 6 V	240 rpm	255 rpm	wheels-up sweep
Motor no-load current	0.16 A	0.197 A	wheels-up sweep
Constant share of friction	0.7	0.56	wheels-up sweep
Winding resistance	4.0 Ω	3.45 Ω	stall
Jetson idle power	2.5 W	1.15 W	rest
Wheel radius	32.5 mm	31.5 mm	ruler
Gearbox efficiency	0.65	0.514	straight and spin
Caster friction	0.25	0.324	spin

One of these deserves a caution. "Gearbox efficiency" moved further than a gearbox plausibly would. It is absorbing drag that belongs to something the twin has no term for.

Two things were deliberately not fitted. Geometry is measured with a ruler, because a fit will distort a wheel to absorb an error that belongs elsewhere. And the driver chip is determined directly, because the fit chose wrongly when asked to infer it.

9 Mistakes, and how they were caught

This section is part of the result. Each item below was a real error in the project, and each was caught by a check that had been built for the purpose, by the robot, or by the person at the bench.

9.1 In the method

Error	How it was caught
The calibration chose the driver chip before fitting anything, and chose wrongly	The rehearsal: the stand-in's chip was known
The calibration was allowed to fit the wheel radius, and returned 10 mm. The held-out score still looked good	The rehearsal: the stand-in's radius was known. **A good held-out score does not mean the parameters are right**
Floor experiments at a command of 0.3, where the stand-in barely moved	The fit stopped responding; the commands were raised clear of the dead band
The "square" had corners near 180°, not 90°	The twin's own prediction, inspected before the run
A hypothesis that the twin's turning inertia was too large	Its uncertainty was computed (about one standard error) and the fit ran to its bound

9.2 In the agent on the robot

Error	How it was caught
It needed a Python I²C library the robot does not have	The probe
It logged at 3.5 Hz instead of 50, because the Jetson's power monitor is slow to read	The first log
It wrote PWM registers as a block, which the chip ignores without a mode bit: the motors never moved	The driver-chip test moved nothing
It timed the motors in the same loop that read the sensors, so a 290 ms turn could last 362 ms	The ten short turns: angle followed duration

9.3 In measuring from video

Error	How it was caught
Wheel speed from the hub's eight identical holes at 30 fps: aliased, and about 5 % low	A slow-motion retake with tape on the hub
Heading from the chassis's long axis: ambiguous by a half-turn; gave 0.75 turns	The answer was physically impossible
Heading by matching each frame to the last: gave 1.8 turns	Two estimates within the tool disagreed
The floor grid followed a shadow's edge: one side of the square came out at 0.91 m and 1.18 m/s	The speed was impossible
A video in which the phone followed the robot	The tool's built-in check failed: joints landed 8 cm from whole tiles. No result was reported

9.4 In the model's geometry

The camera mount took six attempts. The manufacturer's CAD has a fixed bracket; the robot has a hinged one. In turn the model had: the upright passing through the circuit boards; the ribbon cable hanging in mid-air and screws left behind; a thick block where there is a thin sheet; arms too tall for the camera to tilt; the camera 11 mm too high after a misleading photograph. Each was pointed out from a photograph of the real robot.

The response each time was to add a check so the same error could not pass silently again. The build now runs nineteen and writes nothing if one fails.

9.5 At the bench

Event	Consequence
The pack was nearly flat when first measured	Its resistance was fitted at twice the charged value; refitted after charging
The robot lost power abruptly, once, while idle and charging	Unexplained. The log shows no shutdown; candidates are a cell or switch contact, or the protection chip
Tape left on the left tyre through the first floor tests	Noticed at the bench. Tests repeated; the tape turned out to have been hiding the motor mismatch

10 What the twin is good for

10.1 Where it can be trusted

- **Electrical budget and margins.** Rail voltages under load, including the sag at the Jetson, once the 5 V path resistance is included.
- **Steady motion.** How fast the wheels turn for a given command, how far the robot goes in a given time, how fast it spins: to a few percent.
- **Transient timing.** How long the wheels take to spin up and to stop.
- **Design questions.** What happens with the other driver chip, a flat pack, a stalled motor, a failed part.

10.2 Where it cannot

- **Open-loop paths with short turns.** The robot does not repeat them, so no model can predict a single one. The honest output is a distribution.
- **Straight-line drift.** Not modelled. One number would add it.
- **The top of the speed range.** The real motor falls away near full command for a reason not yet identified.
- **A pack that is not fully charged.** Its resistance changes and the twin's does not.

10.3 The general lesson

The twin's failures were more informative than its successes. That a model predicts a straight line is unremarkable. That a model with the right masses, the right motor and the right friction still misses a quarter-turn by 13 %, and that the robot itself varies by three times as much after driving as from

rest, says something specific about where the physics of a cheap differential-drive robot stops being captured by a lumped model: in the first third of a second after a change of direction.

11 Limits of this work

- One robot, one floor, one day. Nothing here says how another JetBot would behave.
- Most quantities were measured once or twice. The exceptions are the short turn (24 times) and the wheels-up sweep (twice).
- The twenty-four short turns were made with the tape on. The tape made little difference to the spin, but that has not been shown for short turns.
- Motor current was never measured directly. Every statement about current is an inference from voltage through a fitted pack resistance.
- The robot's mass was never weighed. The twin uses a sum of estimated parts.
- The Gazebo export was checked only for being well-formed.

12 What to do next

1. **Separate the two explanations for the corners.** A turn after a long pause against a turn straight after a side, and right turns as well as left.
2. **Add the motor mismatch**, from more than one straight run.
3. **A caster model whose drag depends on how long it has been sliding**, to see whether one model can match both the short turn and the long spin.
4. **Pack resistance as a function of charge.**
5. **Holds at 0.85, 0.9 and 0.95**, to see whether the bend at the top of the speed range is gradual or sudden.
6. **Weigh the robot.**
7. **Repeat the short turns without the tape.**

Appendix A: the experiment definitions

Name	Profile: (seconds, left, right)
`rest`	(20, 0, 0)
`jetson_load`	(10, 0, 0), (20, 0, 0) with the CPUs loaded, (10, 0, 0)
`wheels_up_sweep`	ten holds of 2 s at 0.1, 0.15, 0.2, 0.25, 0.3, 0.4, 0.5, 0.6, 0.8, 1.0, each followed by 1 s of rest
`stall`	(2, 0, 0), (1.5, 0.5, 0.5), (3, 0, 0), wheels held
`straight`	(1, 0, 0), (2, 0.5, 0.5), (2, 0, 0)
`spin`	(1, 0, 0), (2, −0.4, 0.4), (2, 0, 0)
`square`	four times: (1.0, 0.5, 0.5), (0.6, 0, 0), (0.29, −0.3, 0.3), (0.6, 0, 0)
`square_small`	as `square` with sides of 0.6 s
`short_turns`	ten times: (0.29, −0.3, 0.3), (1.2, 0, 0)
`short_turns_24`	the same, twenty-four times

Appendix B: where the evidence is

Claim	File
Predictions made beforehand	`results/predictions.txt`, `results/2026-10-04/prediction_*.json`
The rehearsal	`results/rehearsal/rehearsal.txt`
Logs from the robot	`data/real/2026-10-04/*.csv`
Hand measurements	`data/real/2026-10-04/measurements.toml`
Runs made with tape on the tyre	`data/real/2026-10-04/tape_on/`
Wheel speeds from video	`data/real/2026-10-04/wheel_speed_slowmo.json`
Spin and turn angles from video	`spin_no_tape_video.json`, `short_turns_24_video.json`
Square paths	`results/2026-10-04/square_result.json`, `square_small_result.json`
Fitted values	`results/calibrated.json`, `results/2026-10-04/floor_fit.json`
The full running account	`docs/RESULTS.md`

Appendix C: sources

- Waveshare JetBot AI Kit wiki, schematic and 3D drawing.
- `jetbot/motor.py`, NVIDIA-AI-IOT/jetbot.
- Datasheets: TI DRV8870, Toshiba TB6612FNG, NXP PCA9685, TI INA219, Anpec APW7313, AOS AO4407A, ABLIC S-8254A.